

Optimal Strategies for an Obstacle-Avoidance Game on a Triangular Lattice

Presenting author: Jiaming Chu (褚家铭), Team Noxus, Shanghai Xiangming High School **Event:** Long-term Problem 1 of the 5th Shanghai Municipal Secondary-School Mathematics Academic Exhibition (defense scheduled for August 2026) **Repository folder:** `problem1-triangular-grid-steering/` **Companion document:** the complete Chinese report is `problem1_report_zh.pdf` in this directory. This English document is the readable main body—it presents the core mathematics and the verification methodology; the Chinese PDF additionally carries the full 200+ page code appendices.

Abstract

On a triangular lattice of side length 20 m, circular obstacles of radius 8 m are placed at every lattice point except the origin, arranged in concentric hexagonal layers (the k -th layer is a concentric hexagon holding $6k$ disks). A robot of radius 1 m starts at the origin and must escape outward. At every instant it must keep a clearance of at least 9 m from every disk centre (tangency is allowed). At any point of its path the robot may play “steering cards,” each of which changes the heading by at most 5° , with multiple cards usable at the same position. We answer the three subproblems:

- **Subproblem 1** (escaping the first two layers): exactly **9** steering cards.
- **Subproblem 2** (escaping n layers): $N(1) = 0$, $N(n) = 4n + 1$ for $n \geq 2$.
- **Subproblem 3** (lattice rotating at $6^\circ/\text{s}$, with speed-up $\times 1.25$ and slow-down $\times 0.8$ cards, 100 cards in total): final answer 447.9733 m (11 speed-ups + 88 steering + 1 slow-down; zero official validation errors; 43/43 segment Lipschitz certification with clearance lower bound 9.000006), the best-known representable plan in the fixed-chain endgame family.

A single constant underlies the first two subproblems and the static window of the third: the key angle

$$\beta = \arcsin(9/10) - 60^\circ \approx 4.16^\circ (\approx 4.158^\circ).$$

Because adjacent disk centres in one layer are 20 m apart while the required clearance is 9 m, the straight headings that are feasible at all are confined to narrow “windows,” and β is precisely the half-width (equivalently the per-layer turning slack) those windows leave. It is the same essential constant behind the “each layer costs 4β ” bound of Subproblems 1-2 and the static window of Subproblem 3.

Methodologically the work is layered, and the boundary of each layer is declared honestly. Classical geometry pins the upper and lower bounds for Subproblems 1 and 2. A **Lean 4** machine check guards the *combinatorial* layer of the Subproblem 2 lower bound—the implication “five geometric axioms $\Rightarrow N(n) \geq 4n + 1$ ”—while the geometric justification of those five axioms (and of the opaque predicate **Escaped**) is carried by the classical proof. Numerical certification guards the *computational* layer of Subproblem 3, whose proof hierarchy is graded honestly into strict / conditionally-strict-plus-numerical / purely-numerical, with the open items stated as such.

1. Problem & Model

1.1 The game

The plane is tiled by a regular triangular lattice of side 20 m. At every lattice point except the origin a disk obstacle of radius 8 m is placed. A robot of radius 1 m starts at the origin. The robot and an obstacle overlap exactly when the centre-to-centre distance drops below $8 + 1 = 9$ m, so the feasibility condition is: at every instant, the robot's distance to every disk centre is ≥ 9 m, with equality (tangency) allowed.

The obstacles are organised into concentric hexagonal *layers*. The k -th layer is the set of lattice points at hexagonal-norm distance k from the origin; it contains exactly $6k$ disks. The robot's goal is to escape outward through the layers without ever violating the clearance condition.

The robot may steer only by “steering cards”: one card changes the heading by at most 5° , and any number of cards may be played consecutively at the same position (so an effective turn of up to $m \cdot 5^\circ$ costs m cards). The three subproblems ask:

1. **(Q1)** What is the minimum number of cards needed to escape the first *two* layers?
2. **(Q2)** What is the minimum number $N(n)$ of cards needed to escape the first n layers, for arbitrary n ?
3. **(Q3)** The lattice now rotates rigidly about the origin at $6^\circ/\text{s}$ (period 60 s). The robot additionally has speed-up cards ($\times 1.25$) and slow-down cards ($\times 0.8$), and a total budget of 100 cards of any mix. Initial speed is 10 m/s. What is the maximum distance reached *before the first collision*?

1.2 Notation

- $\beta = \arcsin(9/10) - 60^\circ \approx 4.16^\circ$ —the key angle (see below).
- $\alpha \approx 26.74^\circ$ —the heading of the first straight segment in the Q1 upper-bound chain; the first-layer straight window is $[\alpha, 60^\circ - \alpha]$.
- Q_k —an “anchor point” used in the Q2 per-layer construction.
- For Q3: θ_1 is the initial heading, θ_d the dynamically tightened window, Δ_1 the endgame turn angle, L_1 the endgame straight-flight distance.

1.3 The key angle β and the “windows”

Fix one disk of a layer and consider a straight line at heading θ passing near it. Because the clearance is 9 m and adjacent centres in the layer are 20 m apart, the set of straight headings that can thread a gap between two neighbouring disks is a narrow angular *window*: any heading outside the window brings the line within 9 m of one of the two centres. To compute its width, take two neighbouring disks of a layer. Their centres are 20 m apart, so the midpoint of the segment joining them is 10 m from each centre. A common tangent to the two disks —the “door” through the gap —is perpendicular to the radius drawn to the point of tangency; the right triangle formed by a centre, the midpoint of the centre line, and the tangency point has hypotenuse 10 m and opposite leg 9 m (the clearance). The angle at the centre is therefore $\arcsin(9/10) \approx 64.16^\circ$. The lattice direction between the two centres occupies 60° of this, leaving a half-width of

$$\beta = \arcsin \frac{9}{10} - 60^\circ \approx 4.158^\circ,$$

so the full gap subtends an angular window of width $2\beta \approx 8.32^\circ$.

This single constant recurs throughout:

- In Q1/Q2 the per-layer turning cost is measured in units of 4β (four windows' worth of slack per layer), and the second turn of the Q1 chain is exactly 2β .
 - In Q3 the *static* feasibility window of the rotating lattice is again governed by β .
-

2. Subproblem 1: Exactly 9 Cards

Answer: the minimum number of steering cards to escape the first two layers is **9**.

2.1 Upper bound: an explicit three-segment common-tangent chain

We exhibit a path that uses exactly 9 cards and is feasible at every instant (three tangencies, minimum clearance 9.00000 m):

1. **First segment:** fly straight at heading $\alpha \approx 26.74^\circ$ to tangency.
2. **First turn:** turn right by $\alpha + \beta \approx 30.90^\circ$ (cost $\lceil 30.90^\circ/5^\circ \rceil = 7$ cards).
3. **Second turn:** turn by $2\beta \approx 8.32^\circ$ (cost $\lceil 8.32^\circ/5^\circ \rceil = 2$ cards).

The three straight segments are common tangents, and the chain touches the obstacles at three points with minimum clearance exactly 9.00000 m. Hence $7 + 2 = 9$ cards suffice.

2.2 Lower bound: three direction-window lemmas

The lower bound rests on three “direction-window” lemmas:

1. **First-layer straight window.** A path that crosses the first layer in a single straight segment must have heading in $[\alpha, 60^\circ - \alpha]$ (a window of width $60^\circ - 2\alpha$). The parameter $\alpha \approx 26.74^\circ$ is fixed by the geometry of the six disks of the first layer: it is the heading of the straight line that is tangent to the disks bounding the gap, i.e. the edge of the feasible cone.
2. **Corridor lemma.** After the robot has crossed the corridor between the first and second layers, its heading must satisfy heading $\leq -\beta$. Intuitively, once it has slipped through the first gap and is travelling along the corridor, it must be “aimed down” by at least the window half-width relative to the corridor direction, or it will graze the next disk.
3. **Exit lemma.** At the moment it exits the second layer, its heading must satisfy heading $\geq \beta$: a heading below $+\beta$ would not carry it clear of the outer disks of layer 2.

These three windows, pairwise incompatible unless enough turning is available, drive the whole lower bound:

- **0 turns is impossible:** a straight path must simultaneously satisfy the first-layer window $[\alpha, 60^\circ - \alpha]$ and the (disjoint) second-layer straight window; the two straight windows have empty intersection, so no heading threads both layers straight.
- **1 turn is impossible:** after the single turn the corridor lemma forces heading $\leq -\beta$, while the exit lemma forces heading $\geq \beta$. Both would have to hold at the same time, which is impossible since $-\beta < +\beta$. Hence one turn cannot both enter the corridor and leave layer 2.
- **2 turns:** with exactly two turns the first must carry the heading from the first-layer window down to a corridor-compatible value, a net rotation of at least $\approx 30.85^\circ$ (cost $\lceil 30.85^\circ/5^\circ \rceil = 7$ cards); the second must then restore a corridor value up to the exit value, a net rotation of at least $2\beta \approx 8.32^\circ$ (cost $\lceil 8.32^\circ/5^\circ \rceil = 2$ cards). The total variation lower bound is therefore exactly $7 + 2 = 9$.

Therefore 9 cards are necessary and sufficient.

2.3 Numerical corroboration

An independent numerical grid search over candidate paths agrees with the closed-form answer: the optimal candidate achieves a minimum clearance of exactly 9.00000 m, and an independent verification reproduces the same conclusion.

3. Subproblem 2: $N(n) = 4n + 1$

Answer:

n	1	2	3	4	5	6
$N(n)$	0	9	13	17	21	25

i.e. $N(1) = 0$ and $N(n) = 4n + 1$ for $n \geq 2$.

3.1 Upper bound: a per-layer “double-tangency” detour

We construct a feasible path that spends exactly 4 cards per layer beyond the first two, for a total of $9 + 4(n - 2) = 4n + 1$ cards. After the 9-card prefix clears the first two layers (Subproblem 1), each subsequent layer is cleared by an independent 4-card manoeuvre, so the layers do not interact and the costs simply add.

Each additional layer is crossed by a “double-tangency” detour whose two turns together cost

$$2(\beta + \delta) \approx 17.42^\circ, \quad \delta \approx 4.55^\circ,$$

so that 4 cards (each $\leq 5^\circ$) cover it. The extra slack δ over the naive 2β accounts for the need to dip and recover around the disk that blocks the straight corridor at this layer. The detour is engineered so that the path is tangent to the disk at lattice point $(k, 1)$ (clearance = 9), passes through the anchor point Q_k with clearance 9, and keeps a distance of ≈ 9.06 from the disk at $(k, 0)$ —i.e. it is feasible at every instant, with the tightest touchpoints at exactly 9 and a small positive margin (≈ 0.06) at the $(k, 0)$ disk. Iterating this construction over the $n - 2$ layers beyond the first two yields

$$9 + 4(n - 2) = 4n + 1.$$

The tightness of the 4-card count comes from every touchpoint being saturated at the boundary value 9 (or within a controlled margin ≈ 0.06), so no layer can be cleared with fewer than 4 cards without breaking tangency.

3.2 Lower bound: prefix, per-layer 4β , and the fold-back partition

The lower bound is assembled from four ingredients.

Prefix theorem. Before the first escape of layer 2 (at time t_2), the closed interval $[0, t_2]$ contains at least 9 cards —Subproblem 1, in prefix form. (This is Axiom A in the Lean formalisation; see §4.)

Per-layer 4β bound. Crossing one layer requires a net turn of at least 4β . The classical analysis splits into cases according to which side of the blocking disk the path passes on:

- **Corridor / upper-side detour (dive + rise):** the path dives down by 2β and then rises back by 2β , for a net turn of $\geq 2\beta + 2\beta = 4\beta$. This is the tight case, and it is exactly the case the upper-bound construction attains.
- **Lower-side detour:** the path has to go the long way around the disk, a net turn of $\geq 128.3^\circ$ — far larger, hence not the tight case and irrelevant to the matching lower bound.
- **General single-pair bound:** for a pair of disks whose tangent directions differ by θ_{12} , the net turn is bounded below by $2(\theta_{12} + 2\beta)$, which evaluates to one of $\{4\beta, 136.6^\circ, 256.6^\circ\}$; these three values correspond to $\theta_{12} = 0^\circ, 60^\circ, 120^\circ$ respectively (the three possible relative orientations of adjacent lattice disks).

Since each card turns at most 5° , a per-layer turn of $\geq 4\beta \approx 16.63^\circ$ forces ≥ 4 cards per layer.

Global reduction. A global reduction argument (classical, §3-4 of the draft `Q2_global_reduction_draft.md`, audited across six rounds) reduces the general path to the case analysed by the per-layer bound. This reduction is *not* machine-checked; it is a classical result.

Fold-back partition. Let t_j denote the first instant at which the robot escapes layer j . The time axis is partitioned into the closed prefix $[0, t_2]$ and the left-open-right-closed per-layer segments $(t_j, t_{j+1}]$ for $j = 2, \dots, n-1$. These segments cover the relevant interval without overlap and without gap, and each segment satisfies its own bound independently. Summing the prefix (≥ 9) and the $n-2$ per-layer segments (≥ 4 each) gives

$$N(n) \geq 9 + 4(n-2) = 4n + 1,$$

which matches the upper bound, so $N(n) = 4n + 1$. ■

4. Lean 4 Machine Verification

The *combinatorial* layer of the Subproblem 2 lower bound —everything from the five geometric axioms to the theorem $N(n) \geq 4n + 1$ —is machine-verified in Lean 4. This section describes exactly what is and is not machine-checked.

4.1 Target theorem

```
theorem N_lower_bound (n : ℕ) (hn : 2 ≤ n) :
  ∀ p : Plan, Valid n p → (∃ t, Escaped p n t) → p.cards ≥ 4 * n + 1
```

4.2 The time-parameterised model (`Model.lean`)

A `Plan` is an initial heading plus a list of steering-card steps $(\Delta\theta, \tau)$, where τ is the activation time of the card and the times are strictly increasing. The card count is definitionally `p.cards = p.steps.length` (a definitional equality, which the bookkeeping lemmas rely on). The trajectory is reconstructed as a piecewise-linear path from the origin at unit speed, and `Valid n p` asserts that the path keeps distance ≥ 9 from every lattice point in layers $1..n$ at all times.

The discrete model is bound to the game statement by an explicit checklist (`README.md`, §6): constant speed normalised to 1; steering cards bounded by 5° ; tangency ($= 9$) treated as non-collision; and the lattice indexed by the hexagonal norm $\max(|m|, |n|, |m+n|)$, so that the k -th layer is exactly the set of lattice points at hex-norm k . `Escaped` is deliberately kept opaque: its geometric meaning (“the

plan has escaped the k -th fence by time t ”) is stated, not encoded, so that the combinatorial proof is independent of any particular geometric formalisation of “escape.”

Three interval-counting predicates record cards by activation time:

- `CrossCards p t1 t2` —the half-open interval $[t_1, t_2)$ (kept for reference);
- `CrossCardsLe p t1 t2` —the **closed** interval $[t_1, t_2]$ (used by Axiom A);
- `CrossCardsOC p t1 t2` —the **left-open-right-closed** interval $(t_1, t_2]$ (used by Axiom B).

`Escaped` is an **opaque** predicate (its geometric meaning is declared, not defined in Lean): `Escaped p k t` means “plan p has escaped the k -th fence by time t .” `FirstEscape p k t` is the derived predicate “escaped at t , and not before t .”

4.3 The five geometric axioms (`Axioms.lean`)

Exactly five axioms are declared (all fence-geometry facts of classical origin, taken as hypotheses; none is proved in Lean):

Axiom	Statement (semantic form)	Role
A <code>base_nine</code>	$\forall p, t :$ Valid $2 p \wedge \text{FirstEscape } p \ 2 \ t \Rightarrow$ <code>CrossCardsLe</code> $p \ 0 \ t \geq 9$	Subproblem 1 as a <i>closed-prefix</i> bound
B <code>layer_four</code>	$\forall p, k, t_1, t_2 : 2 \leq$ $k \wedge \text{Valid } (k+1) \ p \wedge$ $\text{FirstEscape } p \ k \ t_1 \wedge$ $\text{FirstEscape } p \ (k+1) \ t_2 \Rightarrow$ <code>CrossCardsOC</code> $p \ t_1 \ t_2 \geq 4$	per-layer ≥ 4 cards, on the left-open-right-closed segment
C <code>escape_nested</code>	$\text{Escaped } p \ (k+1) \ t \Rightarrow$ $\text{Escaped } p \ k \ t$	fence nesting
D <code>first_exists</code>	$(\exists t, \text{Escaped } p \ k \ t) \Rightarrow$ $(\exists t, \text{FirstEscape } p \ k \ t)$	first crossing exists
E <code>first_order</code>	$\text{FirstEscape } p \ (k+1) \ t_2 \wedge$ $\text{FirstEscape } p \ k \ t_1 \Rightarrow t_1 < t_2$	outer fence is crossed strictly later

4.4 The proofs (`Counting.lean`, `Main.lean`)

- **Counting.lean** proves, with no placeholders, the bookkeeping layer: a self-contained ceiling function `ceilNat` : $\mathbb{R} \rightarrow \mathbb{N}$ (existence via Archimedean property, minimality via `Nat.find`) with monotonicity and sub-additivity; the fact that each card contributes ≤ 1 to $\sum \lceil |\Delta\theta|/5^\circ \rceil$; validity monotonicity; and the additivity of `CrossCards` (half-open) and `CrossCardsOC` (left-open-right-closed) over adjacent intervals, together with the key no-overlap lemma `crossCardsLe_oc_pair_le` that sums the closed prefix $[0, t]$ with the open-closed chain $(t, t']$.
- **Main.lean** proves, by strong induction, the full target theorem. It first builds the strictly increasing chain of first-escape instants $t_2 < t_3 < \dots < t_n$: `firstEscape_chain` uses Axiom D to obtain each `FirstEscape`, Axiom E to order them, and Axiom C (via `escaped_down`) to descend from an outer escape to all inner escapes. It then applies Axiom A to the closed prefix $[0, t_2]$ (the base case, ≥ 9 cards) and Axiom B to each left-open-right-closed layer segment $(t_j, t_{j+1}]$ (each ≥ 4 cards). The per-layer bounds are summed along the chain by `ccChain_ge_four`; `ccChain_telescope` collapses the sum of adjacent open-closed intervals into the single interval $(t_2, t_n]$; and `crossCardsLe_oc_pair_le` combines the closed prefix with the open-closed chain without double-counting. The final step is the arithmetic $9 + 4(n - 2) = 4n + 1$, closed by `omega`.

4.5 Verification evidence

The acceptance evidence is recorded in `lean_logs.txt`:

- `lake build` completes with **zero errors** (8670 jobs).
- Zero occurrences of `sorry` / `admit`; exactly **5** axiom declarations (A-E); exactly **1** opaque (`Escaped`) —all greps scoped to `*.lean`.
- `#print N_lower_bound` matches the design document verbatim.
- `#print axioms N_lower_bound` outputs exactly the five geometric axioms plus the three logical/library constants:

```
[N_lower_bound] depends on axioms: [base_nine, escape_nested, first_exists,  
first_order, layer_four, propext, Classical.choice, Quot.sound]
```

Environment (reproducible): Lean 4.33.0-rc1; mathlib at commit 8baa3d095e9735e43cf5985d10ae3e9a0f5834d7 (with the dependency pins recorded in `README.md`).

4.6 The v4.4 $\rightarrow$ v4.5 revision (a real counterexample)

The design went through nine audit rounds (v1 $\rightarrow$ v4.5). The final round changed the interval convention, and the reason is a genuine counterexample, not cosmetic.

In v4.4, Axiom A used the **half-open** prefix `CrossCards p 0 t  $\geq$  9`. An independent audit found a legal path that falsifies this: a “door-chord large-turn” path descends along the corridor line and reaches the door chord at $s^* = 9.13$, where a **single** turn of 34.2° is executed that takes effect **exactly at** the first-escape time t (the card is applied on the chord and exits immediately). In the half-open prefix $[0, t)$ this card is *not* counted, so the prefix contains only 7 cards < 9 . The earlier bilateral-window argument quoted in v4.4 (eq (eq:exitwin)) only rules out $\pm\beta$ chord turns, *not* large chord turns.

The fix, adopted as v4.5: Axiom A counts the **closed** prefix `CrossCardsLe p 0 t` (the card effective exactly at t is included), and Axiom B counts the **left-open-right-closed** layer segment `CrossCardsOC p t1 t2` (the card at the segment’s right endpoint —a chord turn —belongs to that layer). The closed prefix and the open-closed chain still partition without overlap and without gap, so the sum $9 + 4(n - 2)$ is unchanged.

4.7 Honest boundary of the machine verification

The machine-checked statement is the **combinatorial implication**

$$(\text{Axioms A-E}) \implies (N(n) \geq 4n + 1 \text{ for all } n \geq 2).$$

The five axioms themselves, and the geometric meaning of the opaque predicate `Escaped`, are **not** machine-proved: their justification is carried by the classical geometry (the direction-window lemmas, the per-layer 4β analysis, and the global reduction). We do **not** claim a machine proof of the *entire* Subproblem 2 theorem; the classical proof and the Lean proof together form the complete argument.

5. Subproblem 3: Rotating Grid, 447.9733 m

5.1 Model

The lattice now rotates rigidly about the origin at $\omega = 6^\circ/\text{s}$ (period 60 s). Equivalently, one works in a co-rotating frame: applying the rigid rotation $R(-\omega t)$ to the robot’s position makes the lattice static,

so that between two consecutive cards the robot’s effective motion is a straight segment. Collision detection therefore reduces to, at each instant, finding the nearest static lattice point in the rotating frame. The robot starts at speed 10 m/s, may use speed-up ($\times 1.25$) and slow-down ($\times 0.8$) cards in addition to steering cards, has 100 cards in total (any mix, applied at arbitrary times, taking effect instantaneously), and the objective is the supremum of $|p(t)|$ over the trajectory up to the first collision. Because the rotation is rigid and the lattice has mirror symmetry, the sense of rotation does not change the achievable distance, although it does affect whether a specific submitted plan collides; the official scorer implements the “rotation about the origin” convention.

5.2 Answer

Quantity	Value
Final answer (max distance before first collision)	447.9733 m
Card budget	11 speed-up + 88 steering + 1 slow-down = 100
Initial heading θ_1	28.0439°
Cruise speed (after 11 speed-ups)	$10 \times 1.25^{11} = 116.4153$ m/s
Flight time to first collision	3.982131 s
Certification: per-segment Lipschitz	43/43 segments SAFE
Minimum clearance lower bound	9.000006 (segment 42, tight disk (22,0); 6 μ m positive margin)
First-collision interval	[447.9661, 447.9818] (covers the official score)
Independent simulator	447.9791 (rerun 447.9792; +0.0058 vs official, within tolerance)

The official scorer (cxk.py) reports zero validation errors. An independent simulator reproduces the score to within 0.0058 m.

5.3 Construction of the plan

The plan is a fixed-structure chain:

1. **11 speed-up cards** at the start, raising speed to the cruise value 116.4153 m/s.
2. A **6-card initial dive**, entering and crossing the first ring (ring 1) at the earliest instant.
3. A **per-ring “dive pair + climb pair” cycle of 4 cards per ring, repeated 20 times** (80 cards), crossing rings 2 through 21: each ring dives toward the band’s inner edge and climbs back, keeping clearance ≥ 9 while advancing one layer. Together with the 6-card dive, the plan thus crosses **21 rings in total** with 86 steering cards.
4. A **2-card endgame turn** $\Delta_1 = 9.8305^\circ$ (two cards, $5.0000^\circ + 4.8305^\circ$).
5. A **straight flight** of length $L_1 = 11.0595$ m.
6. The **1 slow-down card** ($\times 0.8$).
7. The **endgame ray**, flown until first collision.

The steering budget is therefore $6 + 80 + 2 = 88$ cards, and the total is $11 + 88 + 1 = 100$.

5.4 Representability and the cliff phenomenon

The official format imposes a 0.0001 s time grid. At cruise speed this quantises L_1 to a minimum step of

$$116.4153 \text{ m/s} \times 0.0001 \text{ s} = 11.64 \text{ mm.}$$

The continuous optimum of the endgame parameters is $(\Delta_1^*, L_1^*) = (9.832764^\circ, 11.052704 \text{ m})$, which is **not representable** in the official format; the submitted plan uses the nearest representable values $\Delta_1 = 9.8305^\circ$, $L_1 = 11.0595 \text{ m}$.

The endgame exhibits a **cliff**: at $\Delta_1 = 9.8304^\circ$ the robot collides at 439.50 m, while at $\Delta_1 = 9.8305^\circ$ it escapes to 447.97 m —a score jump of 8.47 m across a 0.0001° step. The full family contains 203 **cliff edges** of this kind, which is why the representable optimum must be resolved to the last grid point.

5.5 Proof hierarchy (graded honestly)

The Subproblem 3 argument is graded into three tiers, and we state each tier’s status exactly:

- **Strict (rigorous)**: the ring-band topology of the rotating lattice; the static window governed by β and the band radius ρ ; the initial-angle bound $\theta_1 \geq 26.74^\circ$; and the intra-family arithmetic showing that 101 cards would exceed the budget by exactly 1.
- **Conditionally strict + numerical ($\approx 97\%$)**: the per-ring lower bound of ≥ 4 cards (dynamic tightening $\theta_d \approx 4.26^\circ$, verified numerically by three independent routes) and the entry requirement of ≥ 6 cards.
- **Purely numerical**: the claim that reaching $525.x \text{ m}$ would need ≥ 8 speed-up cards, and the exhaustive search showing that 3 cards per ring admits zero feasible plans (1954 combinations enumerated).

5.6 Open items (declared as such)

- There is **no global proof** that distances $> 447.9733 \text{ m}$ are unreachable.
- The $\approx 97\%$ -confidence conclusions of §5.5 have **not** been upgraded to strict results.
- The 0.0001° full enumeration (about 190,000 candidate plans) has only run to $\approx 19\%$ completion (the run is checkpointed and resumable; completion is estimated at about one week).

Consequently the correct label for the result is: **the best-known representable plan within the fixed-chain-tail endgame family**, *not* a global optimum. All conclusions of the full report were cross-checked by a five-way parallel independent audit.

6. Methodology & Honesty Notes

The three subproblems are held up by three complementary methods, and each method’s boundary is stated rather than implied:

1. **Geometric proof** fixes the upper and lower bounds of Subproblems 1 and 2, and closes them to the exact values 9 and $4n + 1$.
2. **Machine verification** (Lean 4) guards the *combinatorial* layer of the Subproblem 2 lower bound: the derivation of $N(n) \geq 4n + 1$ from five explicitly-declared geometric axioms. The axioms and the opaque predicate **Escaped** are the interface; their geometric truth is classical. Together the classical proof and the Lean proof constitute the complete proof.
3. **Numerical certification** guards the *computational* layer of Subproblem 3: static-window strictness, dynamically tightened windows (three-way check), endgame-family optimisation, and per-segment Lipschitz certification of all 43 segments. Every non-strict conclusion is labelled with its tier, and the open items (§5.6) are stated explicitly.

The layering is principled rather than incidental. Subproblems 1 and 2 admit exact closed forms, so their entire argument belongs to classical geometry and machine-checkable combinatorics. Subproblem

3, by contrast, is a continuous-time, rotating, finite-budget optimisation whose optimum is *not* a closed form; there the honest standard is not “proved optimal” but “certified to a declared confidence tier,” with the gap to a global optimality proof stated openly. In every case the boundary between what is proved, what is machine-checked, and what is numerically certified is written down, never left implicit.

All conclusions of the full report were additionally cross-checked by a five-way parallel independent audit, and every issue the audit raised was corrected in the revised version. No claims are fabricated or inflated beyond these declared boundaries.

7. Appendix Pointers: Code and Logs

All code, certification scripts, and run logs ship with the repository. The layout (relative to this folder) is:

```
problem1-triangular-grid-steering/
├─ lean-formalization/          # Lean 4 project (Subproblem 2 lower bound, combinatorial layer)
├─ python-q1-q2/                # Python verification scripts (Subproblems 1–2)
└─ python-q3-certification/      # Q3 official scorer, answer, and certification scripts
```

lean-formalization/

File	Content
Model.lean	Plan, Escaped, FirstEscape, trajectory, Valid, CrossCards/CrossCardsLe/CrossCards0C
Axioms.lean	the five geometric axioms A–E
Counting.lean	ceilNat, validity monotonicity, interval additivity, bookkeeping (fully proved)
Main.lean	target theorem + strong induction (fully proved)
Q2Formal.lean	top-level import aggregator
README.md	boundary statement + hand-verification checklist
lean_logs.txt	build log and #print axioms evidence
Q2_lean_baseline_plan.md	design document (v1 → v4.5)
Q2_global_reduction_draft.md	classical global-reduction draft

python-q1-q2/

File	One-line purpose
analyze_q1.py	Subproblem 1 analysis and candidate search
appendix_search.py	numerical grid search for optimal candidates
appendix_verify.py	verification of the searched candidates
verify_exit_lemma.py	independent check of the exit (direction-window) lemma
verify_q2.py	Subproblem 2 per-layer and prefix verification
verify_drift.py	drift / bound verification
tu.py	shared turn & geometry utilities

python-q3-certification/

File	One-line purpose
cxk.py	official scorer (format validation + rotating-frame simulator)
q3_campaign_best_saA.txt	the submitted answer file (447.9733 m)
q3_certp1_*.py	Part 1 certification (static window / initial-angle alignment)
q3_certp2_*.py	Part 2 certification (chain construction / segment cross-checks)
q3_certp3_*.py + q3_certp3_cliff.json	Part 3 certification (per-segment bounds, cliff edges)
q3_certp4_*.py	Part 4 exhaustive enumeration (resumable)
q3_audit_sim.py	independent simulator
Q3_*.md, report_q3_*.md, q3_audit_*.md	audit reports and memos

The Python sources are intentionally *not* reproduced in this document; they are provided verbatim in the repository.

Appendix A: Lean 4 Source

The complete source of the formalisation is reproduced below verbatim. (Files `lakefile.toml`, `lean-toolchain`, and the two `.md` design documents are in the repository and omitted here for brevity.)

Q2Formal.lean

```
import Q2Formal.Model
import Q2Formal.Axioms
import Q2Formal.Counting
import Q2Formal.Main
```

Q2Formal/Model.lean

```
/-
第二问下界 " $N(n) \geq 4n+1$  ( $\forall n \geq 2$ )" 组合层形式化 ——离散模型
(设计文档 `Q2_lean_baseline_plan.md` v4.4 第 1 节, 逐字遵守)
```

注: 设计终版 v4.4 中, 公理 A 为前缀形式 (见 `Axioms.lean` 的 `base_nine`)。
语义绑定: 见 `README.md` 第 6 节对应清单; 本文件不引用任何第三问文件。

```
-/
import Mathlib
```

noncomputable section

open scoped Classical

/-- 计划: 初始航向 + 转向卡列表。`steps` 的每个元素 (θ_i, τ_i) 是第 i 张转向卡的

转角与生效时刻, τ 严格递增 (题面: 每张卡转角 $\leq 5^\circ$, 语义绑定见 README 第 1 条)。
 路径由原点出发、时刻 θ 起、速度归一为 1。 -/

```

structure Plan where
  heading0 : ℝ
  steps    : List (ℝ × ℝ)

namespace Plan

/-- 卡数。设计文档第 1 节记 `cards : ℕ -- = steps.length`; 此处以定义式访问器实现,
    使 `p.cards = p.steps.length` 为定义相等 (记账引理 Counting.lean 直接使用)。 -/
def cards (p : Plan) : ℕ := p.steps.length

end Plan

/-- 不透明谓词 (语义绑定 README 第 6.2 条): "p 在时刻 t 已逃出第 k 层栅栏" -/
opaque Escaped : Plan → ℕ → ℝ → Prop

/-- 首次逃出谓词 (v4.2: Classical.choose 只能取任意见证, 故"首次"必须谓词化) -/
def FirstEscape (p : Plan) (k : ℕ) (t : ℝ) : Prop :=
  Escaped p k t ∧ ∀ s < t, ¬ Escaped p k s

/-- 方向角  $\theta$  的单位方向向量 -/
def dirVec ( $\theta$  : ℝ) : ℝ × ℝ := (Real.cos  $\theta$ , Real.sin  $\theta$ )

/-- 平面向量加法 -/
def vecAdd (a b : ℝ × ℝ) : ℝ × ℝ := (a.1 + b.1, a.2 + b.2)

/-- 平面向量数乘 -/
def vecSmul (r : ℝ) (a : ℝ × ℝ) : ℝ × ℝ := (r * a.1, r * a.2)

/-- 轨迹重建: 按  $\tau_i$  分界逐段直线; 第 i 段方向 = heading0 +  $\sum_{j<i} \theta_j$ , 速度归一为 1;
    分界点即 steps 的时刻。 -/
def pathPoint (p : Plan) (t : ℝ) : ℝ × ℝ :=
  let rec go (pos : ℝ × ℝ) (heading : ℝ) (time : ℝ) : List (ℝ × ℝ) → ℝ × ℝ
    | [] => vecAdd pos (vecSmul (t - time) (dirVec heading))
    | ( $\theta, \tau$ ) :: rest =>
      if  $\tau \leq t$  then
        go (vecAdd pos (vecSmul ( $\tau$  - time) (dirVec heading))) (heading +  $\theta$ )  $\tau$  rest
      else
        vecAdd pos (vecSmul (t - time) (dirVec heading))
  go ( $\theta, \theta$ ) p.heading0 0 p.steps

/-- 格点  $m e_1 + n e_2$ , 其中  $e_1 = (20, 0)$ ,  $e_2 = (10, 10\sqrt{3})$  -/
def latticePoint (m n : ℤ) : ℝ × ℝ :=
  ((20 : ℝ) * (m : ℝ) + (10 : ℝ) * (n : ℝ), (10 : ℝ) * Real.sqrt 3 * (n : ℝ))

/-- 六边形范数  $\max(|m|, |n|, |m+n|)$  ——第 k 层的层号 -/
def layerIndex (m n : ℤ) : ℕ :=
  max (Int.natAbs m) (max (Int.natAbs n) (Int.natAbs (m + n)))

```

```

/-- 第 k 层格点集 = { m e1 + n e2 | max(|m|, |n|, |m+n|) = k } (k ≥ 1) -/
def Layer (k : ℕ) : Set (ℝ × ℝ) :=
  { c | ∃ m n : ℤ, c = latticePoint m n ∧ layerIndex m n = k }

/-- 两点距离 -/
def planDist (a b : ℝ × ℝ) : ℝ := Real.sqrt ((a.1 - b.1) ^ 2 + (a.2 - b.2) ^ 2)

/-- 分层 Valid: 轨迹全程与第 1..n 层格点距离 ≥ 9 -/
def Valid (n : ℕ) (p : Plan) : Prop :=
  ∀ k : ℕ, 1 ≤ k → k ≤ n → ∀ t : ℝ, ∀ c : ℝ × ℝ, c ∈ Layer k → planDist (pathPoint p t) c ≥ 9

/-- 区间卡数 (v4.2): 半开区间 [t1, t2) 内的转向卡数 -/
def CrossCards (p : Plan) (t1 t2 : ℝ) : ℕ :=
  (p.steps.filter (fun st => t1 ≤ st.2 ∧ st.2 < t2)).length

/-- 区间卡数 (闭): [t1, t2] 内的转向卡数。
    v4.5 修订: 公理 A 改用本定义 (前缀闭区间化) ——第二问弦上大转角转向
    可恰发生在首次逃出第 2 层的时刻, 该卡必须计入前缀 9 卡。 -/
def CrossCardsLe (p : Plan) (t1 t2 : ℝ) : ℕ :=
  (p.steps.filter (fun st => t1 ≤ st.2 ∧ st.2 ≤ t2)).length

/-- 区间卡数 (左开右闭): (t1, t2] 内的转向卡数。
    v4.5 修订: 公理 B 改用本定义 ——逐层段右端 = 首次逃出时刻, 该时刻的卡
    (含弦上转向) 归入本层段, 与闭前缀 [0, t0] 拼合无重叠、无遗漏。 -/
def CrossCardsOC (p : Plan) (t1 t2 : ℝ) : ℕ :=
  (p.steps.filter (fun st => t1 < st.2 ∧ st.2 ≤ t2)).length

```

Q2Formal/Axioms.lean

```

/-
组合层公理 ——恰 5 条 (设计文档 `Q2_lean_baseline_plan.md` v4.4 第 2 节; 签名逐字照抄, 不增删)。
其中公理 A 为前缀形式 (v4.5 修订: 闭区间): `base_nine` 断言第 2 层首次逃出时刻 t 及之前的闭区间 [0, t] 卡数

五条公理 A-E 均为栅栏几何的古典事实, 同源同地位: 显式声明、不在 Lean 内证明。
docstring 指向 report.tex / Q2_global_reduction_draft.md 的精确位置; 不引用任何第三问文件。
-/
import Q2Formal.Model

/-- 公理 A = 第一问 (report.tex Theorem 1, 第 341-343 行; 下界论证第 369-391 行:
    净转角 [30.85°/5°] + [8.32°/5°] = 7 + 2 = 9)。
    前缀局部化形式 (v4.5 修订: 闭区间): 第 2 层首次逃出时刻 t 及之前的卡数 ≥ 9。
    古典真值: 第一次转向 (≥30.85°, 7 卡) 严格先于 t; "走廊 → 出射" 的第二次转向 (≥2β, 2 卡)
    必发生在 ≤ t 时刻 ——直线穿行时严格 < t, 恰在门缝弦上转向时 = t。v4.4 的半开形式
    在 "弦上大转角转向" 路径下为假 (合法反例: 走廊下降线直抵门缝弦 s* = 9.13 处一次
    转向 34.2° 穿出, 该卡恰在 t 生效, 半开前缀仅 7 卡 < 9), 故 v4.5 改为闭区间
    CrossCardsLe, 恰在 t 生效的卡计入前缀。 -/
axiom base_nine : ∀ (p : Plan) (t : ℝ), Valid 2 p → FirstEscape p 2 t → CrossCardsLe p 0 t ≥ 9

```

```

/-- 公理 B = 逐层穿越  $\geq 4$  卡 (v4.2: 按 FirstEscape 首次时刻陈述, 显式参数版;
v4.5 修订: 区间由半开  $[t_1, t_2)$  改为左开右闭  $(t_1, t_2]$  —— 一段右端  $t_2 =$  首次逃出
第  $k+1$  层的时刻, 该时刻的卡 (含弦上转向) 归入本层段; 每层  $4\beta$  转向 (下降 + 回升,
或弦上大转角) 全部落在  $(t_1, t_2]$  内; 与闭前缀  $[0, t_0]$  拼合无重叠、无遗漏。
古典背书: report.tex 第 459-470 行门到门  $4\beta$  + 草稿 v0.3 第 3 节绕行  $\geq 4\beta$ )。 -/
axiom layer_four :  $\forall (p : \text{Plan}) (k : \mathbb{N}) (t_1 \ t_2 : \mathbb{R}), 2 \leq k \rightarrow \text{Valid } (k+1) \ p \rightarrow$ 
    FirstEscape p k  $t_1 \rightarrow$  FirstEscape p (k+1)  $t_2 \rightarrow$ 
    CrossCards0C p  $t_1 \ t_2 \geq 4$ 

/-- 公理 C = 栅栏嵌套 (不透明谓词 Escaped 的古典事实; README 第 6.2 条绑定)。 -/
axiom escape_nested :  $\forall (p : \text{Plan}) (k : \mathbb{N}) (t : \mathbb{R}),$ 
    Escaped p (k+1)  $t \rightarrow$  Escaped p k t

/-- 公理 D = 古典事实 F1 (v4.3 由 README 事实升级为公理; 栅栏几何, 与 C 同源):
首次穿越时刻存在 (折线 + 闭栅栏)。 -/
axiom first_exists :  $\forall (p : \text{Plan}) (k : \mathbb{N}), (\exists t, \text{Escaped } p \ k \ t) \rightarrow$ 
     $(\exists t, \text{FirstEscape } p \ k \ t)$ 

/-- 公理 E = 古典事实 F2 (同源): 外栅栏首次逃出严格晚于内栅栏 (连续性)。 -/
axiom first_order :  $\forall (p : \text{Plan}) (k : \mathbb{N}) (t_1 \ t_2 : \mathbb{R}),$ 
    FirstEscape p (k+1)  $t_2 \rightarrow$  FirstEscape p k  $t_1 \rightarrow t_1 < t_2$ 

```

Q2Formal/Counting.lean

```

/-
记账引理 (设计文档 `Q2_lean_baseline_plan.md` v4.4 第 4.1 节; 全证, 无占位)。
注意: 设计终版 v4.5 的公理 A 为闭区间前缀形式 (`CrossCardsLe`), 公理 B 为左开右闭
逐层段 (`CrossCards0C`), 本文件引理与之配套 (半开 `CrossCards` 引理保留备用)。
ceil 单调 / 次可加; 卡数  $\geq \Sigma$  段转角/ $5^\circ$ ; 区间可加 (半开/左开右闭); Valid n  $\rightarrow$  Valid k。
-/
import Q2Formal.Model

noncomputable section

open scoped Classical

/-- 上取整 ceil :  $\mathbb{R} \rightarrow \mathbb{N}$  (自证版: 存在性 = 阿基米德性 `exists_nat_ge`,
最小性 = `Nat.find`)。 -/
def ceilNat (x :  $\mathbb{R}$ ) :  $\mathbb{N} := \text{Nat.find } (\text{exists\_nat\_ge } x)$ 

/-- ceil 主性质:  $x \leq \text{ceilNat } x$  -/
theorem ceilNat_spec (x :  $\mathbb{R}$ ) :  $x \leq (\text{ceilNat } x : \mathbb{R}) :=$ 
    Nat.find_spec (exists_nat_ge x)

/-- ceil 单调 -/
theorem ceilNat_mono {x y :  $\mathbb{R}$ } (h :  $x \leq y$ ) :  $\text{ceilNat } x \leq \text{ceilNat } y := \text{by}$ 
    apply Nat.find_min'
    exact le_trans h (ceilNat_spec y)

```

```

/-- ceil 次可加 -/
theorem ceilNat_add_le (x y : ℝ) : ceilNat (x + y) ≤ ceilNat x + ceilNat y := by
  apply Nat.find_min'
  have hx : x ≤ (ceilNat x : ℝ) := ceilNat_spec x
  have hy : y ≤ (ceilNat y : ℝ) := ceilNat_spec y
  calc
    x + y ≤ (ceilNat x : ℝ) + (ceilNat y : ℝ) := add_le_add hx hy
    _ = ((ceilNat x + ceilNat y : ℕ) : ℝ) := by rw [Nat.cast_add]

/-- 5° (弧度) -/
def deg5 : ℝ := (5 : ℝ) / 180 * Real.pi

theorem deg5_pos : 0 < deg5 := by
  unfold deg5
  positivity

/-- 每张卡至多 5° 时, 单卡对  $\sum \lfloor |\theta|/5^\circ \rfloor$  的贡献  $\leq 1$  -/
theorem ceil_turn_le_one {θ : ℝ} (h : |θ| ≤ deg5) : ceilNat (|θ| / deg5) ≤ 1 := by
  apply Nat.find_min'
  rw [div_le_iff, deg5_pos]
  simp using h

/-- List 版记账: 若每项  $f a \leq 1$ , 则  $(l.map f).sum \leq l.length$  -/
theorem List_sum_le_length {α : Type u} (f : α → ℕ) (l : List α) (h : ∀ a ∈ l, f a ≤ 1) :
  (l.map f).sum ≤ l.length := by
  induction l with
  | nil => simp
  | cons a t ih =>
    have ht : ∀ b ∈ t, f b ≤ 1 := fun b hb => h b (by simp [hb])
    specialize ih ht
    have ha : f a ≤ 1 := h a (by simp)
    simp
    omega

/-- 记账: 卡数  $\geq \sum \lfloor \text{段转角}/5^\circ \rfloor$  (每张卡  $\leq 5^\circ$  时) -/
theorem cards_ge_sum_ceil_turns (p : Plan) (hsmall : ∀ st ∈ p.steps, |st.l| ≤ deg5) :
  p.cards ≥ (p.steps.map (fun st => ceilNat (|st.l| / deg5))).sum := by
  have h : ∀ st ∈ p.steps, ceilNat (|st.l| / deg5) ≤ 1 :=
    fun st hst => ceil_turn_le_one (hsmall st hst)
  simp [Plan.cards] using
    (List_sum_le_length (fun st => ceilNat (|st.l| / deg5)) p.steps h)

/-- Valid 单调: Valid n → Valid k ( $k \leq n$ ) -/
theorem valid_mono {n k : ℕ} {p : Plan} (h : k ≤ n) (hv : Valid n p) : Valid k p := by
  intro j hj1 hjk t c hc
  exact hv j hj1 (le_trans hjk h) t c hc

/-- CrossCards 半开区间可加:  $[t_1, t_2) \sqcup [t_2, t_3) = [t_1, t_3)$  (需  $t_1 \leq t_2 \leq t_3$ ;
  端点归属由定义直接给出; 对卡表逐项分类证明) -/

```

```

theorem crossCards_add (p : Plan) (t1 t2 t3 : ℝ) (ht1 : t1 ≤ t2) (ht2 : t2 ≤ t3) :
  CrossCards p t1 t2 + CrossCards p t2 t3 = CrossCards p t1 t3 := by
  unfold CrossCards
  induction p.steps with
  | nil => simp
  | cons st t ih =>
    by_cases h1 : t1 ≤ st.2 ∧ st.2 < t2
    • by_cases h2 : t2 ≤ st.2 ∧ st.2 < t3
      • have : False := (not_lt_of_ge h2.1) h1.2
        cases this
      • have h3 : t1 ≤ st.2 ∧ st.2 < t3 := ⟨h1.1, lt_of_lt_of_le h1.2 ht2⟩
        simp [h1, h2, h3] at ih ⊢
        omega
    • by_cases h2 : t2 ≤ st.2 ∧ st.2 < t3
      • have h3 : t1 ≤ st.2 ∧ st.2 < t3 := ⟨le_trans ht1 h2.1, h2.2⟩
        simp [h1, h2, h3] at ih ⊢
        omega
      • have h3 : ¬ (t1 ≤ st.2 ∧ st.2 < t3) := by
          intro h
          by_cases hle : t2 ≤ st.2
          • exact h2 ⟨hle, h.2⟩
          • exact h1 ⟨h.1, lt_of_not_ge hle⟩
        simp [h1, h2, h3] at ih ⊢
        omega

/-- 嵌套 □ 可加: 相邻首次逃出时刻给出两段互不重叠的半开区间之和 -/
theorem crossCards_disjoint_add (p : Plan) (t1 t2 t3 : ℝ) (h1 : t1 ≤ t2) (h2 : t2 ≤ t3) :
  CrossCards p t1 t2 + CrossCards p t2 t3 = CrossCards p t1 t3 :=
  crossCards_add p t1 t2 t3 h1 h2

/-- CrossCards ≤ 总卡数 -/
theorem crossCards_le_cards (p : Plan) (t1 t2 : ℝ) : CrossCards p t1 t2 ≤ p.cards := by
  unfold CrossCards Plan.cards
  induction p.steps with
  | nil => simp
  | cons st t ih =>
    by_cases h : t1 ≤ st.2 ∧ st.2 < t2 <=> simp [h] at ih ⊢ <=> omega

/-- 两个互不重叠半开区间上的卡数之和 ≤ 总卡数 (各层区间加总用) -/
theorem crossCards_pair_le (p : Plan) (t1 t2 t3 : ℝ) :
  CrossCards p t1 t2 + CrossCards p t2 t3 ≤ p.cards := by
  unfold CrossCards Plan.cards
  induction p.steps with
  | nil => simp
  | cons st t ih =>
    by_cases h1 : t1 ≤ st.2 ∧ st.2 < t2
    • by_cases h2 : t2 ≤ st.2 ∧ st.2 < t3
      • have : False := (not_lt_of_ge h2.1) h1.2
        cases this

```



```

    • simp [h1, h2] at ih ⊢
      omega
    • by_cases h2 : t₂ ≤ st.2 ∧ st.2 < t₃
      • simp [h1, h2] at ih ⊢
        omega
      • simp [h1, h2] at ih ⊢
        omega

/-- CrossCardsOC 左开右闭区间可加: (t₁,t₂] ∪ (t₂,t₃] = (t₁,t₃] (需 t₁ ≤ t₂ ≤ t₃;
v4.5 修订: 逐层段左开右闭, 端点归属由定义直接给出; 对卡表逐项分类证明) -/
theorem crossCardsOC_add (p : Plan) (t₁ t₂ t₃ : ℝ) (ht₁ : t₁ ≤ t₂) (ht₂ : t₂ ≤ t₃) :
  CrossCardsOC p t₁ t₂ + CrossCardsOC p t₂ t₃ = CrossCardsOC p t₁ t₃ := by
  unfold CrossCardsOC
  induction p.steps with
  | nil => simp
  | cons st u ih =>
    by_cases h1 : t₁ < st.2 ∧ st.2 ≤ t₂
    • by_cases h2 : t₂ < st.2 ∧ st.2 ≤ t₃
      • have : False := (not_lt_of_ge h1.2) h2.1
        cases this
      • have h3 : t₁ < st.2 ∧ st.2 ≤ t₃ := ⟨h1.1, le_trans h1.2 ht₂⟩
        simp [h1, h2, h3] at ih ⊢
        omega
    • by_cases h2 : t₂ < st.2 ∧ st.2 ≤ t₃
      • have h3 : t₁ < st.2 ∧ st.2 ≤ t₃ := ⟨lt_of_le_of_lt ht₁ h2.1, h2.2⟩
        simp [h1, h2, h3] at ih ⊢
        omega
      • have h3 : ¬ (t₁ < st.2 ∧ st.2 ≤ t₃) := by
          intro h
          by_cases hle : t₂ < st.2
          • exact h2 ⟨hle, h.2⟩
          • exact h1 ⟨h.1, le_of_not_gt hle⟩
        simp [h1, h2, h3] at ih ⊢
        omega

/-- 闭前缀 [0, t] 与左开右闭链 (t, t'] 无重叠, 卡数和 ≤ 总卡数 (v4.5 修订:
公理 A 闭前缀 + 公理 B 左开右闭逐层段的合成引理; τ ≤ t 与 t < τ 互斥) -/
theorem crossCardsLe_oc_pair_le (p : Plan) (t t' : ℝ) :
  CrossCardsLe p 0 t + CrossCardsOC p t t' ≤ p.cards := by
  unfold CrossCardsLe CrossCardsOC Plan.cards
  induction p.steps with
  | nil => simp
  | cons st u ih =>
    by_cases h1 : 0 ≤ st.2 ∧ st.2 ≤ t
    • by_cases h2 : t < st.2 ∧ st.2 ≤ t'
      • have : False := (not_lt_of_ge h1.2) h2.1
        cases this
      • simp [h1, h2] at ih ⊢
        omega

```

- by_cases h2 : $t < st.2 \wedge st.2 \leq t$ '
 - simp [h1, h2] at ih $\vdash$
omega
 - simp [h1, h2] at ih $\vdash$
omega

Q2Formal/Main.lean

/-

目标定理 + 强归纳 (设计文档 `Q2\_lean\_baseline\_plan.md` v4.4 第 3-4 节; 全证, 无占位)。

证明骨架: 基例 = 公理 A(闭区间前缀形式 `base\_nine`, v4.5 修订);

归纳步 = 公理 B + D/E + 左开右闭区间可加, 合成 $9 + 4(n-2) = 4n+1$ 。

-/

```
import Q2Formal.Axioms
import Q2Formal.Counting
```

noncomputable section

open Classical

/-- Escaped 向下嵌套: 由公理 C 迭代 (n 降到 k)。 -/

```
lemma escaped_down (p : Plan) (t : ℝ) :
  ∀ {k n : ℕ}, k ≤ n → Escaped p n t → Escaped p k t := by
  intro k n hkn hn
  induction n using Nat.strong_induction_on generalizing k with
  | h n ih =>
    by_cases hk : k = n
    • subst k
      exact hn
    • have hkl : k < n := lt_of_le_of_ne hkn hk
      have hk' : k ≤ n - 1 := by omega
      have hn' : Escaped p (n - 1) t := by
        have h1n : 1 ≤ n := by omega
        have hnn : Escaped p (n - 1 + 1) t := by
          simp [Nat.sub_add_cancel h1n] using hn
        exact escape_nested p (n - 1) t hnn
      exact ih (n - 1) (by omega) hk' hn'
```

/-- 首次逃出链: $t_2 < t_3 < \dots < t_n$, 每层 FirstEscape (由公理 D/E 构造, 强归纳)。 -/

```
lemma firstEscape_chain (p : Plan) : ∀ {n : ℕ}, 2 ≤ n → (∃ t, Escaped p n t) →
  ∃ ts : List ℝ, ts.length = n - 1 ∧
    (∀ k : ℕ, 2 ≤ k → k ≤ n → ∀ hk : k - 2 < ts.length,
      FirstEscape p k (ts[k - 2]'hk)) ∧
    (∀ k : ℕ, 2 ≤ k → k + 1 ≤ n → ∀ hk : k - 2 < ts.length, ∀ hk' : k - 1 < ts.length,
      ts[k - 2]'hk < ts[k - 1]'hk') := by
  intro n hn hesc
  induction n using Nat.strong_induction_on with
  | h n ih =>
```

```

by_cases hn2 : n = 2
• subst n
rcases first_exists p 2 hesc with (t₂, hfe)
refine ([t₂], by simp, ?_, ?_)
• intro k hk2 hk hk'
  have : k = 2 := by omega
  subst k
  simpa using hfe
• intro k hk2 hk hk' hk''
  omega
• have hn1 : 2 ≤ n - 1 := by omega
have hesc1 : ∃ t, Escaped p (n - 1) t := by
  rcases hesc with (t₀, h₀)
  exact (t₀, escaped_down p t₀ (by omega) h₀)
rcases ih (n - 1) (by omega) hn1 hesc1 with (ts', hlen', hFE', hord')
rcases first_exists p n hesc with (tn, hfen)
have hfeprev : FirstEscape p (n - 1) (ts'[(n - 1) - 2]'(by omega)) :=
  hFE' (n - 1) (by omega) (by omega) (by omega)
have hordlast : ts'[(n - 1) - 2]'(by omega) < tn := by
  have h1n : 1 ≤ n := by omega
  have hfenn : FirstEscape p (n - 1 + 1) tn := by
    simpa [Nat.sub_add_cancel h1n] using hfen
  exact first_order p (n - 1) (ts'[(n - 1) - 2]'(by omega)) tn hfenn hfeprev
refine (ts' ++ [tn], ?_, ?_, ?_)
• have : (ts' ++ [tn]).length = (n - 1 - 1) + 1 := by simp [hlen']
  omega
• intro k hk2 hk hk'
by_cases hklt : k < n
• have hk1 : k ≤ n - 1 := by omega
  have hget : (ts' ++ [tn])[k - 2]'hk' = ts'[k - 2]'(by omega) := by
    apply List.getElem_append_left
  rw [hget]
  exact hFE' k hk2 hk1 (by omega)
• have : k = n := by omega
  subst k
  have hget : (ts' ++ [tn])[n - 2]'hk' = tn := by
    have hidx : n - 2 = ts'.length := by omega
    simp only [hidx]
    rw [List.getElem_append_right (by omega : ts'.length ≤ ts'.length)]
    simp
  rw [hget]
  exact hfen
• intro k hk2 hk hk' hk''
by_cases hk1n : k + 1 = n
• have : k = n - 1 := by omega
  subst k
  have h1 : (ts' ++ [tn])[(n - 1) - 2]'hk' = ts'[(n - 1) - 2]'(by omega) := by
    apply List.getElem_append_left
  have h2 : (ts' ++ [tn])[(n - 1) - 1]'hk'' = tn := by

```

```

    have hidx : (n - 1) - 1 = ts'.length := by omega
    simp only [hidx]
    rw [List.getElem_append_right (by omega : ts'.length ≤ ts'.length)]
    simp
    rw [h1, h2]
    exact hordlast
  • have hk1lt : k + 1 < n := by omega
    have h1 : (ts' ++ [tn])[k - 2]'hk' = ts'[k - 2]'(by omega) := by
      apply List.getElem_append_left
    have h2 : (ts' ++ [tn])[k - 1]'hk'' = ts'[k - 1]'(by omega) := by
      apply List.getElem_append_left
    rw [h1, h2]
    exact hord' k hk2 (by omega) (by omega) (by omega)

/-- CrossCards 自交为空: [t, t) 无卡。 -/
lemma crossCards_self (p : Plan) (t : ℝ) : CrossCards p t t = 0 := by
  have h := crossCards_add p t t t (le_refl) (le_refl)
  omega

/-- CrossCards0C 自交为空: (t, t] 无卡 (v4.5: 链望远镜 m=0 情形用)。 -/
lemma crossCards0C_self (p : Plan) (t : ℝ) : CrossCards0C p t t = 0 := by
  have h := crossCards0C_add p t t t (le_refl) (le_refl)
  omega

/-- t ≤ 0 时 [0, t) 无卡。 -/
lemma crossCards_right_nonpos (p : Plan) {t : ℝ} (ht : t ≤ 0) : CrossCards p 0 t = 0 := by
  unfold CrossCards
  rw [List.length_eq_zero_iff, List.filter_eq_nil_iff]
  intro st hst
  intro h
  have hd : 0 ≤ st.2 ∧ st.2 < t := of_decide_eq_true h
  have : st.2 < 0 := lt_of_lt_of_le hd.2 ht
  exact (not_lt_of_ge hd.1) this

/-- 相邻首出时刻对的卡数和: 沿链 t₀ t₁ ... tₘ 累加 CrossCards0C(tᵢ, tᵢ₊₁]
(v4.5: 左开右闭, 段右端首逃时刻的卡归本段)。用结构递归取代 range 索引,
避免 getElem 界证明。 -/
def ccChain (p : Plan) : List ℝ → ℕ
| [] => 0
| [_] => 0
| t₀ :: t₁ :: rest => CrossCards0C p t₀ t₁ + ccChain p (t₁ :: rest)

theorem ccChain_cons (p : Plan) (t₀ t₁ : ℝ) (rest : List ℝ) :
  ccChain p (t₀ :: t₁ :: rest) = CrossCards0C p t₀ t₁ + ccChain p (t₁ :: rest) := rfl

/-- 严格递增链的传递: i < j ≤ m 时 ts[i] < ts[j] (对 j - (i+1) 归纳)。 -/
lemma chain_lt_between {m : ℕ} {ts : List ℝ}
  (hord : ∀ k : ℕ, 0 ≤ k → k + 1 < m + 1 → ∀ hk : k < ts.length, ∀ hk' : k + 1 < ts.length,
    ts[k]'hk < ts[k + 1]'hk') :

```

```

  ∀ i j : ℕ, i < j → j ≤ m → ∀ hi : i < ts.length, ∀ hj : j < ts.length,
    ts[i]'hi < ts[j]'hj := by
intro i j hij hjm hi hj
induction h : j - (i + 1) generalizing i j with
| zero =>
  have hij' : i + 1 = j := by omega
  have hordi := hord i (by omega) (by omega) hi (by omega)
  simpa only [hij'] using hordi
| succ d ih =>
  have h1 : ts[i]'hi < ts[i + 1]'(by omega) := hord i (by omega) (by omega) hi (by omega)
  have h2 : ts[i + 1]'(by omega) < ts[j]'hj := by
    apply ih (i + 1) j
    • omega
    • omega
    • omega
  exact lt_trans h1 h2

```

/-- 望远镜：相邻左开右闭区间卡数和 = 首尾区间（逐对 crossCardsOC_add）。 -/

```

lemma ccChain_telescope (p : Plan) : ∀ (m : ℕ) (ts : List ℝ), ts.length = m + 1 →
  (∀ k : ℕ, 0 ≤ k → k + 1 < m + 1 → ∀ hk : k < ts.length, ∀ hk' : k + 1 < ts.length,
    ts[k]'hk < ts[k + 1]'hk') →
  ∀ (h0 : 0 < ts.length) (hm : m < ts.length),
    ccChain p ts = CrossCardsOC p (ts[0]'h0) (ts[m]'hm) := by
intro m ts hlen hord h0 hm
induction ts generalizing m with
| nil => simp at hlen
| cons t₀ ts' ih =>
  by_cases hm0 : m = 0
  • subst m
    cases ts' with
    | nil =>
      dsimp [ccChain]
      exact (crossCardsOC_self p t₀).symm
    | cons t₁ rest => simp at hlen
  • cases ts' with
    | nil => simp at hlen; omega
    | cons t₁ rest =>
      have hlen' : (t₁ :: rest).length = m - 1 + 1 := by
        have : (t₁ :: rest).length = m := by
          have := hlen
          simp at this
          omega
        omega
      have hord' : ∀ k : ℕ, 0 ≤ k → k + 1 < (m - 1) + 1 →
        ∀ hk : k < (t₁ :: rest).length, ∀ hk' : k + 1 < (t₁ :: rest).length,
          (t₁ :: rest)[k]'hk < (t₁ :: rest)[k + 1]'hk' := by
        intro k hk0 hk hkB
        have h1 : (t₁ :: rest)[k]'hkB = (t₀ :: t₁ :: rest)[k + 1]'(by omega) := rfl
        have h2 : (t₁ :: rest)[k + 1]'hkB = (t₀ :: t₁ :: rest)[(k + 1) + 1]'(by omega) := rfl

```

```

    rw [h1, h2]
    exact hord (k + 1) (by omega) (by omega) (by omega) (by omega)
  have ht0t1 : t0 ≤ t1 := by
    have : (t0 :: t1 :: rest)[0]'(by omega) < (t0 :: t1 :: rest)[1]'(by omega) :=
      hord 0 (by omega) (by omega) (by omega) (by omega)
    change t0 < t1 at this
    exact le_of_lt this
  have ht1t3 : t1 ≤ (t0 :: t1 :: rest)[m]'hm := by
    by_cases hm1 : m = 1
    • subst m
      exact le_refl
    • have h1m : 1 < m := by omega
      have : (t0 :: t1 :: rest)[1]'(by omega) < (t0 :: t1 :: rest)[m]'hm :=
        chain_lt_between hord 1 m h1m (by omega) (by omega) hm
      change t1 < (t0 :: t1 :: rest)[m]'hm at this
      exact le_of_lt this
  have h0' : 0 < (t1 :: rest).length := by omega
  have hm' : m - 1 < (t1 :: rest).length := by omega
  calc
    ccChain p (t0 :: t1 :: rest)
      = CrossCardsOC p t0 t1 + ccChain p (t1 :: rest) := rfl
    _ = CrossCardsOC p t0 t1 + CrossCardsOC p t1 ((t0 :: t1 :: rest)[m]'hm) := by
      rw [ih (m - 1) hlen' hord' h0' hm']
      rw [List.getElem_cons_zero]
      cases m with
      | zero => omega
      | succ m' => rfl
    _ = CrossCardsOC p ((t0 :: t1 :: rest)[0]'h0) ((t0 :: t1 :: rest)[m]'hm) := by
      apply crossCardsOC_add
      • exact ht0t1
      • exact ht1t3

```

-- 逐层 ≥ 4 卡：公理 B 沿链累加（层号 k 随链下移而 +1）。 --

```

lemma ccChain_ge_four (p : Plan) (n k : ℕ) (hv : Valid n p) :
  ∀ (m : ℕ) (ts : List ℝ), ts.length = m + 1 →
  (∀ j : ℕ, 0 ≤ j → j + 1 ≤ m + 1 → ∀ hj : j < ts.length,
    FirstEscape p (j + k) (ts[j]'hj)) →
  2 ≤ k → k + m ≤ n →
  4 * m ≤ ccChain p ts := by
  intro m ts hlen hFE hk2 hkn
  induction ts generalizing m k with
  | nil => simp at hlen
  | cons t0 ts' ih =>
    by_cases hm0 : m = 0
    • subst m
      cases ts' with
      | nil => simp [ccChain]
      | cons t1 rest => omega
    • cases ts' with

```

```

| nil => simp at hlen; omega
| cons t₁ rest =>
  have hlen' : (t₁ :: rest).length = m - 1 + 1 := by
    have : (t₁ :: rest).length = m := by
      have := hlen
      simp at this
      omega
    omega
  have hFE' : ∀ j : ℕ, 0 ≤ j → j + 1 ≤ (m - 1) + 1 →
    ∀ hj : j < (t₁ :: rest).length,
    FirstEscape p (j + (k + 1)) ((t₁ :: rest)[j]'hj) := by
    intro j hj0 hj hj'
    have hget : (t₁ :: rest)[j]'hj' = (t₀ :: t₁ :: rest)[j + 1]'(by omega) := rfl
    rw [hget]
    have hk' : (j + 1) + k = j + (k + 1) := by omega
    rw [← hk']
    exact hFE (j + 1) (by omega) (by omega) (by omega)
  have hih : 4 * (m - 1) ≤ ccChain p (t₁ :: rest) :=
    ih (k + 1) (m - 1) hlen' hFE' (by omega) (by omega)
  have hCC : 4 ≤ CrossCardsOC p t₀ t₁ := by
    apply layer_four p k t₀ t₁
    • exact hk2
    • exact valid_mono (show k + 1 ≤ n from by omega) hv
    • have hf0 := hFE 0 (by omega) (by omega) (by omega)
      rw [List.getElem_cons_zero] at hf0
      simpa using hf0
    • have hf1 := hFE 1 (by omega) (by omega) (by omega)
      rw [show (1 : ℕ) + k = k + 1 from by omega] at hf1
      rw [List.getElem_cons_succ] at hf1
      rw [List.getElem_cons_zero] at hf1
      exact hf1
  rw [ccChain_cons]
  exact le_trans (by omega : 4 * m ≤ 4 + 4 * (m - 1)) (add_le_add hCC hih)

```

-- 目标定理 (设计文档第 3 节, 逐字):

```

N_lower_bound (n : ℕ) (hn : 2 ≤ n) :
  ∀ p : Plan, Valid n p → (∃ t, Escaped p n t) → p.cards ≥ 4 * n + 1 -/
theorem N_lower_bound (n : ℕ) (hn : 2 ≤ n) :
  ∀ p : Plan, Valid n p → (∃ t, Escaped p n t) → p.cards ≥ 4 * n + 1 := by
  intro p hv hesc
  rcases firstEscape_chain p hn hesc with (ts, hlen, hFE, hord)
  let m : ℕ := n - 2
  have hm_len : ts.length = m + 1 := by dsimp [m]; omega
  have hFE0 : ∀ j : ℕ, 0 ≤ j → j + 1 ≤ m + 1 → ∀ hj : j < ts.length,
    FirstEscape p (j + 2) (ts[j]'hj) := by
    intro j hj0 hj hj'
    have hFEj := hFE (j + 2) (by omega) (by omega) (by omega)
    simpa only [show (j + 2) - 2 = j from by omega] using hFEj
  have hord0 : ∀ k : ℕ, 0 ≤ k → k + 1 < m + 1 → ∀ hk' : k < ts.length,

```

```

    ts[k]'hk < ts[k + 1]'hk' := by
intro k hk0 hk hKA hKB
have hordk := hord (k + 2) (by omega) (by omega) (by omega) (by omega)
simp only [show (k + 2) - 2 = k from by omega, show (k + 2) - 1 = k + 1 from by omega] using hordk
-- 基例: 前 9 卡 (公理 A, v4.5 闭前缀)
have hbase : CrossCardsLe p 0 (ts[0]'(by omega)) ≥ 9 :=
  base_nine p (ts[0]'(by omega)) (valid_mono (show 2 ≤ n from hn) hv)
  (hFE0 0 (by omega) (by omega) (by omega))
-- 逐层 ≥ 4 卡 (公理 B)
have hlayers : 4 * m ≤ ccChain p ts :=
  ccChain_ge_four p n 2 hv m ts hm_len hFE0 (by omega) (by omega)
-- 望远镜 + 左开右闭区间可加
have htel : ccChain p ts = CrossCardsOC p (ts[0]'(by omega)) (ts[m]'(by omega)) :=
  ccChain_telescope p m ts hm_len hord0 (by omega) (by omega)
have htotal : CrossCardsLe p 0 (ts[0]'(by omega)) + ccChain p ts ≤ p.cards := by
  rw [htel]
  exact crossCardsLe_oc_pair_le p (ts[0]'(by omega)) (ts[m]'(by omega))
-- 合成  $9 + 4(n-2) = 4n + 1$ 
have hsum :  $9 + 4 * m \leq p.cards$  := le_trans (add_le_add hbase hlayers) htotal
dsimp [m] at hsum ⊢
omega

```